

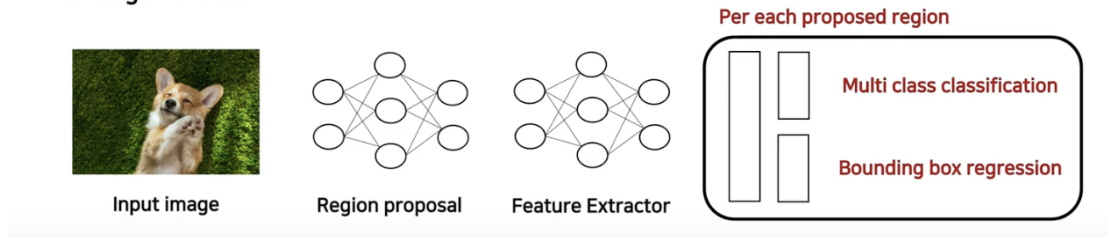
week7

▼ 논문리뷰 _ YOLOv3 : an incremental improvement

사전지식

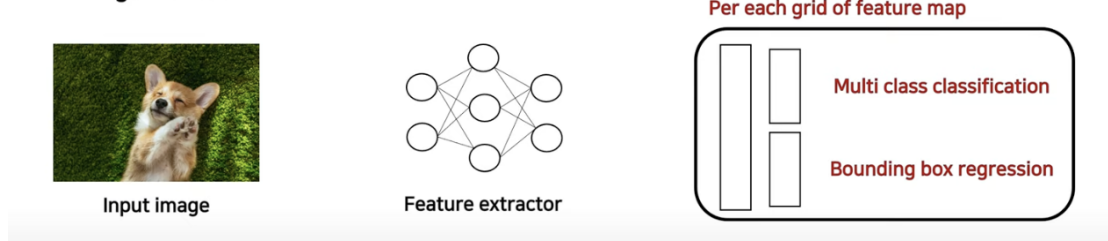
▼ 1, 2 stage detector / YOLO

• 2-stage detector



region proposal과 object분류 과정이 별도의 네트워크로 분리됨 (2개의 stage로 분리됨)
detection의 성능은 상대적으로 높지만 inference 속도가 상대적으로 낮음

• 1-stage detector



region proposal과 object분류 과정이 별도의 네트워크로 분리되지않음 (1개의 stage로)
detection의 성능은 상대적으로 낮지만 inference 속도가 상대적으로 높음

• 1-stage detector 연대기

YOLO v1	SSD	YOLO v2	Feature Pyramid Network	RetinaNet	YOLO v3
2015.06	2015.12	2016.12	2017	2017.08	2018.04
EfficientDet	YOLO v4				
2019.11	2020.04				

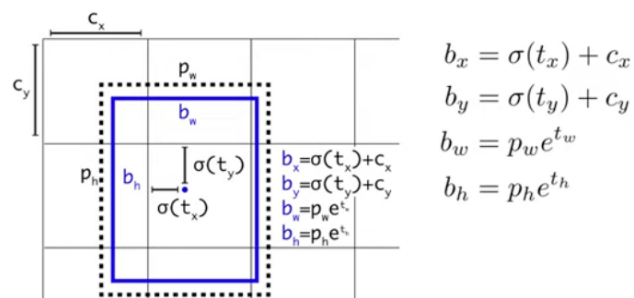
- YOLO의 motivation

YOLO는 정확한 성능보다는 real-time application을 위한 빠른 속도의 알고리즘에 초점을 둠

YOLO v1	빠른 속도, 낮은 정확도
YOLO v2	v1 대비 속도, 성능 개선 (small object 낮은 성능)
YOLO v3	v2 대비 속도는 다소 느리지만, 성능은 개선 (small object 성능 개선)
YOLO v4	v3 대비 속도와 성능 개선

주요 내용

▼ bounding box prediction



p_w, p_h : Anchor box size

t_x, t_y, t_w, t_h : 모델의 예측 offset 값

b_x, b_y, b_w, b_h : 예측 bounding box central coordinates and (width, height)

파란색 표시 : bounding box / 검은색 점선 표시 : anchor box

정규화를 해주어 bounding box가 1을 벗어나지 않도록 크기를 조정해줌

파란색박스의 중점, 가로 세로 값을 구해 최종적으로 파란색박스를 구함!

YOLO v2부터는 anchor box라는 prior box를 설정해 최종 bounding box예측에 활용함

▼ Loss

▼ B-box regression loss

$$\lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{obj} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

B-box 중심 (x, y) part

B-box (width, height) part

$S^2 : (S, S)$ feature map grid 수

$B : \text{Anchor Box 수}$

$\mathbb{I}_{ij}^{obj} : \text{cell } i \text{ 의 responsible } j \text{ th bounding box predictor}$ responsible : has the highest IOU of any predictor in that grid cell

$\lambda_{coord} : \text{Bbox regression loss의 가중치 (보통 5로 설정)}$

- 예측한 B-box와 Groundtruth B-box의 오차 제곱을 계산
- 1이라고 표시된 indicate function → B-box에 대해서만 연산하도록 함
⇒ loss연산과 업데이트를 효율적으로 하기 위함
- B-box width, height 부분에서 제곱근은 절대적인 값이 커짐으로 오류가 커지게 되는 왜곡을 막기위해 사용
- 람다라는 가중치를 둬으로써 스케일링을 진행 ← 일종의 균형을 맞춰주기위한 balance parameter

▼ object confidence loss

✓ Object Confidence Loss

$$+ \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{noobj} \left(C_i - \hat{C}_i \right)^2 + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{obj} \left(C_i - \hat{C}_i \right)^2$$

$S^2 : (S, S)$ feature map grid 수

$B : \text{Anchor Box 수}$

$\mathbb{I}_{ij}^{obj} : \text{cell } i \text{ 의 responsible } j \text{ th bounding box predictor}$ responsible : has the highest IOU of any predictor in that grid cell

$\lambda_{noobj} : \text{responsible 하지 않은 Bbox의 loss에 대한 가중치 (보통 0.5설정)}$

$\hat{C}_i : P(\text{object}) \times \text{IOU}$, $C_i : \text{Ground truth confidence score '1'}$

- object 확률값 x iou값 - Groundtruth과의 오차제곱을 계산
없어야하는 배경에대해서도 점수를 구해 계산에 포함시킴
람다 가중치를 붙여줌 보통 0.5임 → 클래스 불균형 문제를 해소시키기위함(물체보다 배경에 집중되지않게하기위해서)

▼ classification loss

✓ Classification loss

$$+ \sum_{i=0}^{S^2} \mathbb{I}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

$S^2 : (S, S)$ feature map grid 수

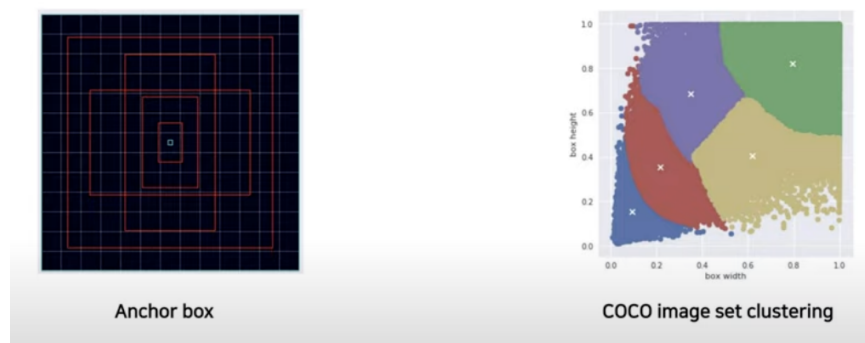
$\mathbb{I}_{ij}^{\text{obj}} : \text{cell } i \text{ 의 responsible } j \text{ th bounding box predictor}$ responsible : has the highest IOU of any predictor in that grid cell

$p_i(c) : \text{해당 cell } i \text{ 내 존재하는 Object가 } c \text{ class 일 확률}$

클래스별 확률값에 대한 오차 제곱으로 계산

▼ 과정

▼ bounding box prediction (with anchor box)



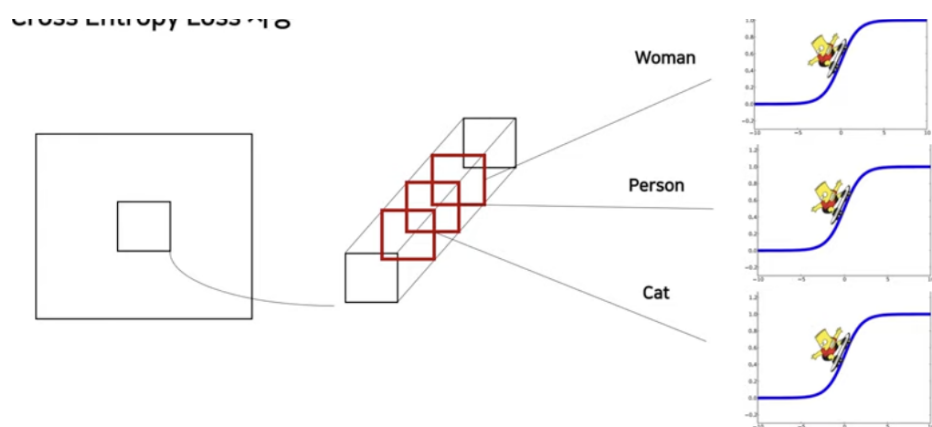
coco dataset으로 진행한 경우

1개의 cell에서 여러개의 anchor box를 통해 개별 cell에서 여러개의 object detection 을 진행

anchor box의 크기는 데이터셋 이미지의 object 크기를 clustering하여 계산

v3에서는 총 9개로 clustering함

▼ binary prediction per every class



logistic classifier를 사용함 (하나로만 가는 softmax classifier를 사용하지 않음)

binary crss entropy loss를 사용함

▼ feature extractor

□ Darknet - 53

	Type	Filters	Size	Output
	Convolutional	32	3 × 3	256 × 256
	Convolutional	64	3 × 3 / 2	128 × 128
1x	Convolutional	32	1 × 1	
	Convolutional	64	3 × 3	
	Residual			128 × 128
	Convolutional	128	3 × 3 / 2	64 × 64
2x	Convolutional	64	1 × 1	
	Convolutional	128	3 × 3	
	Residual			64 × 64
	Convolutional	256	3 × 3 / 2	32 × 32
8x	Convolutional	128	1 × 1	
	Convolutional	256	3 × 3	
	Residual			32 × 32
	Convolutional	512	3 × 3 / 2	16 × 16
8x	Convolutional	256	1 × 1	
	Convolutional	512	3 × 3	
	Residual			16 × 16
	Convolutional	1024	3 × 3 / 2	8 × 8
4x	Convolutional	512	1 × 1	
	Convolutional	1024	3 × 3	
	Residual			8 × 8
	Avgpool		Global	
	Connected		1000	
	Softmax			

• Darknet-53

: YOLO v2의 backbone(Darknet-19)에 residual network 요소를 더함

bottle neck구조로, short-cut을 도입함

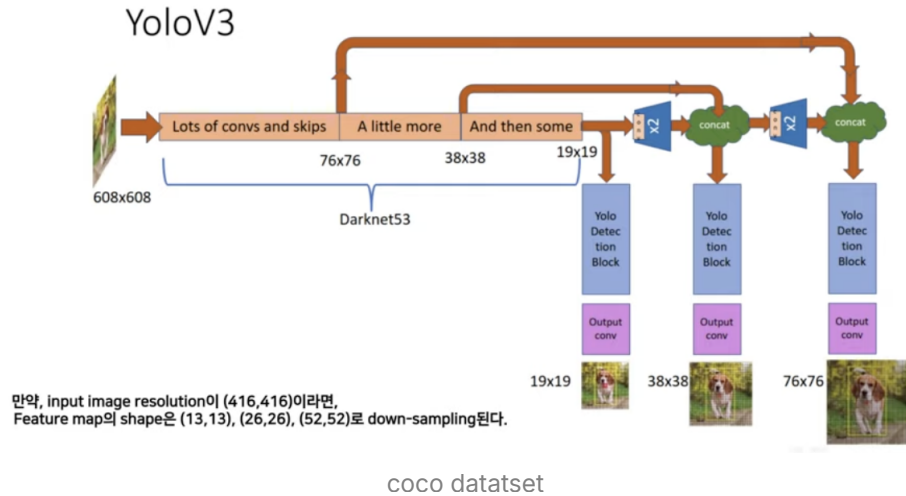
총 53개의 convolution layer를 사용함

Backbone	Top-1	Top-5	Bn Ops	BFLOP/s	FPS
Darknet-19 [100]	74.1	91.8	7.29	1246	171
ResNet-101 [55]	77.1	93.7	19.7	1039	53
ResNet-152 [55]	77.6	93.8	29.4	1090	37
Darknet-53	77.2	93.8	18.7	1457	78

YOLO v2의 backbone(Darknet-19)보다 높은 성능을 지님

SOTA(Resnet 101, Resnet 152) 구조 대비 대등한 성능 및 효율적인 연산

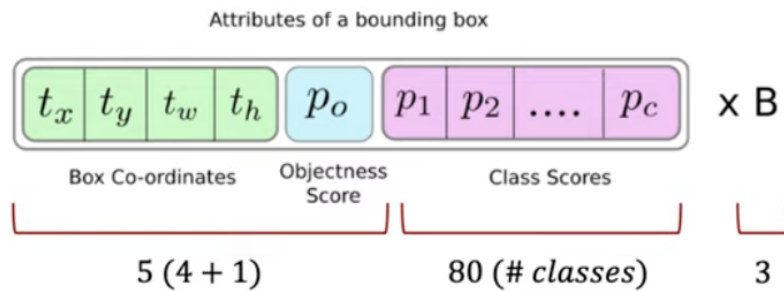
▼ YOLO v3



input 이미지 → feature map(down sampling) → 19×19 detection → 2배로 사이즈업
→ detection → 2배로 사이즈업 → detection

성능을 높일 수 있도록 디자인되어짐

▼ output feature map



✓ Output Feature map shape "(13,13) scale case"

- 1) Pascal VOC : (13, 13, (20 + 5)×3), (26,26, (20 + 5)×3), (52,52, (20 + 5)×3) #class : 20
- 2) COCO : (13, 13, (80 + 5)×3), (26,26, (80 + 5)×3), (52,52, (80 + 5)×3) #class : 80

pascal voc, coco dataset

Box Co-ordinates : bounding box의 x, y, w, h 값

Objectness score : object일 확률 x IOU 값

class score : 각 class 별 score 값

⇒ Box Co-ordinates x objectness score x class scores x bounding box 수

▼ Experiment

	backbone	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
<i>Two-stage methods</i>							
Faster R-CNN++ [5]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [8]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [6]	Inception-ResNet-v2 [21]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [20]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	52.1
<i>One-stage methods</i>							
YOLOv2 [15]	DarkNet-19 [15]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [11, 3]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [3]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [9]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [9]	ResNeXt-101-FPN	40.8	61.1	44.1	24.1	44.2	51.2
YOLOv3 608 × 608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

Table 3. I'm seriously just stealing all these tables from [9] they take soooo long to make from scratch. Ok, YOLOv3 is doing alright. Keep in mind that RetinaNet has like 3.8× longer to process an image. YOLOv3 is much better than SSD variants and comparable to state-of-the-art models on the AP₅₀ metric.

SSD계열인 DSSD와 유사한 AP성능을 보이면서도, 속도는 3배 빠름

APs(: small object에 대한 정확도)에 대한 성능이 v2에 비해 약 3배 높아짐

RetinaNet을 제외한 다른 모델에 대해서 AP성능이 대부분 높거나, 상단을 이루는 것을 확인

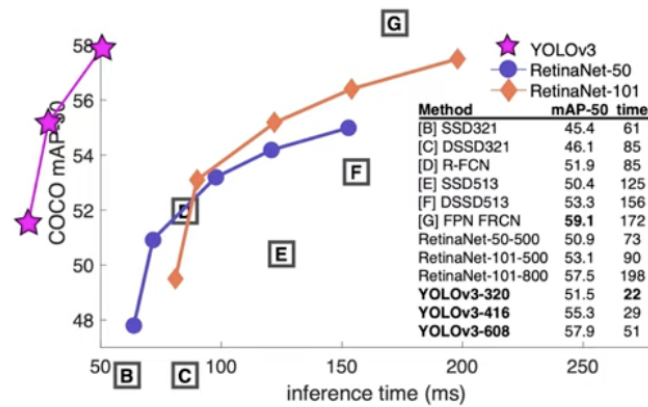


Figure 3.3: Again adapted from the [78], this time displaying speed/accuracy tradeoff on the mAP at .5 IOU metric. You can tell YOLOv3 is good because it's very high and far to the left. Can you cite your own paper? Guess who's going to try, this guy → [101]. Oh, I forgot, we also fix a data loading bug in YOLOv2, that helped by like 2 mAP. Just sneaking this in here to not throw off layout.

IOU 0.5 metric에 대해, 정확도는 유사하면서 추론 속도가 매우 높은 것을 확인

▼ 의미

항목	V1	V2	V3
Feature Extractor	Inception 변형	Darknet 19	Darknet 53
Grid당 Anchor Box 수	2개	5개	9개 (Feature map 3개 x 다른 scale 3개)
Anchor Box 결정 방법		K-Means Clustering	K-Means Clustering
Output Feature Map 사이즈		13 x 13	13 x 13, 26 x 26, 52 x 52 3개의 Feature Map 사용
Feature Map scaling			Feature Pyramid Network

multi scale(3개)의 feature map output에서 각각 서로 다른 크기의 anchor box로 detection 진행 → 보다 다양한 크기의 object에 대한 detection 성능 향상

backbone 성능 향상 : Darknet - 53

multi label prediction : softmax가 아닌 logistic classifier로 binary prediction 문제로 치환

1개의 이미지에 대해서 총 9개의 anchor box를 지니게 됨

▼ 논문리뷰 _ YOLOv4 : Optimal Speed and Accuracy of Object Detection

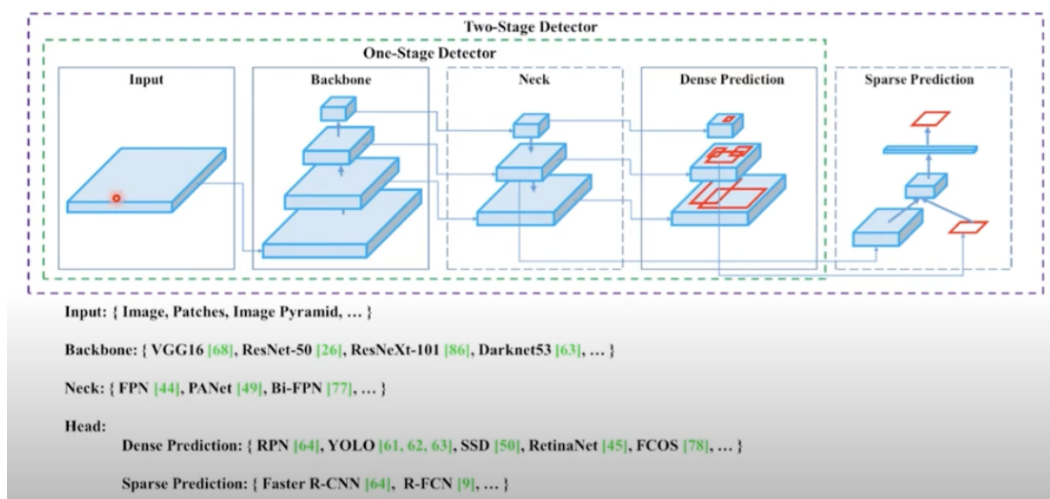
▼ Background

joseph redmon, YOLO v1~ YOLO v3, YOLO 9000

이제는 더이상 CV연구를 하지 않겠다고 하심

▼ Related works

- object detection



backbone : pretrained된 모델들을 주로 사용함(darknet53)

neck : backbone과 head부분을 연결해줌. feature에대한 아키텍처

head : one stage detection / two stage detection 나누는 부분

- BoF(bag of freebies)
: 효과적이고, 효율적인 학습을 위한 기법들

Data augmentation

Regularization method

Normalization(batch normalization)

Object Function

- BoS(bag of specials)

: 정확도 향상을 위한 추가 모듈 기법 및 후처리 방법

Enlarging Receptive Field

Attention mechanism

feature integration

▼ Methodology

- YOLO v4 모델

Table 1: Parameters of neural networks for image classification.

Backbone model	Input network resolution	Receptive field size	Parameters	Average size of layer output (WxHxC)	BFLOPs (512x512 network resolution)	FPS (GPU RTX 2070)
CSPResNext50	512x512	425x425	20.6 M	1058 K	31 (15.5 FMA)	62
CSPDarknet53	512x512	725x725	27.6 M	950 K	52 (26.0 FMA)	66
EfficientNet-B3 (ours)	512x512	1311x1311	12.0 M	668 K	11 (5.5 FMA)	26

backbone으로 Darknet 53 사용

작은 물체를 탐지하기위해 더 큰 해상도 이미지를 사용

큰 해상도 이미지를 다루기위해 더 많은 layer로 큰 receptive field

다양한 크기의 물체를 탐지하기위해 더 큰 capacity 모델

CSP(cross stage partial) : CSPNet에서 제안된 방법으로, 보다 더 효율적인 연산을 할 수 있는 아키텍처 구조

▼ BoF, BoS

- **Activations**: ReLU, leaky-ReLU, parametric-ReLU, ReLU6, SELU, **Swish, or Mish**
- **Bounding box regression loss**: MSE, IoU, GloU, CloU, **DIoU**
- **Data augmentation**: CutOut, MixUp, **CutMix**
- **Regularization method**: DropOut, DropPath, Spatial DropOut, or **DropBlock**
- **Normalization of the network activation by their mean and variance**: Batch Normalization (BN), Cross-GPU Batch Normalization (CGBN or SyncBN), Filter Response Normalization (FRN), or **Cross-Iteration Batch Normalization (CBN)**
- **Skip-connections**: Residual connections, Weighted residual connections, Multi-input weighted residual connections, or **Cross stage patial connections (CSP)**

여러 실험을 진행하였고, 결국에는 초록색으로 표시된 것들을 사용하게 됨

▼ 추가적인 방법으로...

- **Data Augmentation:** Mosaic, Self-Adversarial Training (SAT)
 - **Hyperparameter Search:** Genetic Algorithm
 - **Modified Existing Methods:** Modified SAM, Modified PAN, and Cross mini-Batch Normalization (CmBN)
- Mosaic : 여러가지 이미지를 붙여서 한 장의 이미지로 만드는 방법



4장의 이미지를 한장으로 합친것이기에.. batchsize를 4로 한 효과가 존재

- Modified SAM, PAN



Figure 5: Modified SAM.

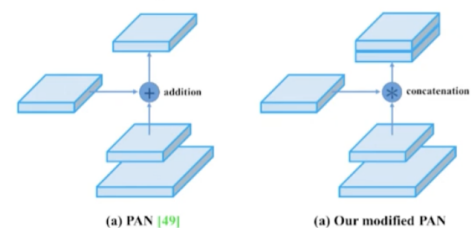


Figure 6: Modified PAN.

- Modified SAM : 이미지 → max pooling, average pooling → convolution으로 하나의 이미지로 → sigmoid를 씌워 0~1까지 값을 갖게됨 → 원래 이미지와 곱해줌 (attention 적용)

← point 단위로 계산해서 사용하였음

- Modified PAN : 기존의 feature map을 더한 방식 → feature map을 concat한 방식

- Cross mini-batch normalization

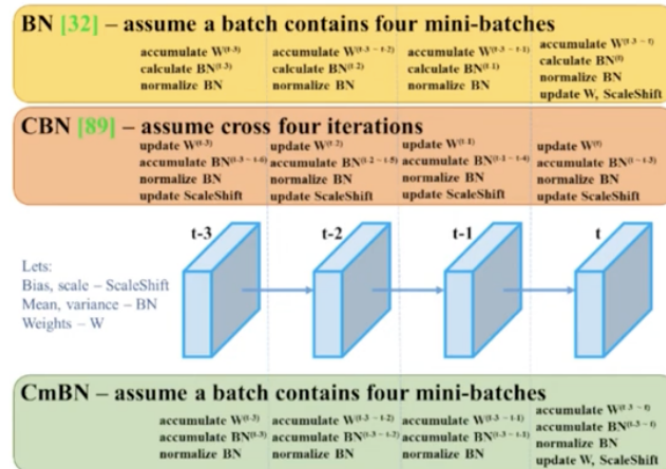


Figure 4: Cross mini-Batch Normalization.

CmBN : 기존의 batch를 더 작게 batch로 나누어서 적용한 것

- Backbone : CSPDarknet 53 모델을 사용
- Neck : SPP, PAN 방법들을 사용
- Head : YOLOv3와 동일한 것을 사용

backbone에서와 detector에서 각각 사용된 것을 볼 수 있음

YOLO v4 uses:

- **Bag of Freebies (BoF) for backbone:** CutMix and Mosaic data augmentation, DropBlock regularization, Class label smoothing
- **Bag of Specials (BoS) for backbone:** Mish activation, Cross-stage partial connections (CSP), Multi-input weighted residual connections (MiWRC)
- **Bag of Freebies (BoF) for detector:** CIoU-loss, CmBN, DropBlock regularization, Mosaic data augmentation, Self-Adversarial Training, Eliminate grid sensitivity, Using multiple anchors for a single ground truth, Cosine annealing scheduler [52], Optimal hyperparameters, Random training shapes
- **Bag of Specials (BoS) for detector:** Mish activation, SPP-block, SAM-block, PAN path-aggregation block, DIoU-NMS

▼ Experiments

Table 2: Influence of BoF and Mish on the **CSPResNeXt-50** classifier accuracy.

MixUp	CutMix	Mosaic	Blurring	Label Smoothing	Swish	Mish	Top-1	Top-5
							77.9%	94.0%
✓							77.2%	94.0%
	✓						78.0%	94.3%
		✓					78.1%	94.5%
			✓				77.5%	93.8%
				✓			78.1%	94.4%
					✓		64.5%	86.0%
	✓	✓		✓		✓	78.9%	94.5%
	✓	✓		✓		✓	78.5%	94.8%
				✓		✓	79.8%	95.2%

Table 3: Influence of BoF and Mish on the **CSPDarknet-53** classifier accuracy.

MixUp	CutMix	Mosaic	Blurring	Label Smoothing	Swish	Mish	Top-1	Top-5
							77.2%	93.6%
	✓	✓		✓			77.8%	94.4%
	✓	✓		✓		✓	78.7%	94.8%

- classifier training

2가지 모델로 계속해서 실험함

CSPResNeXt-50

CSPDarknet-53

⇒ CutMix, Mosaic, Label Smoothing, Mish 적용했을때 가장 좋은 성능을 보임

▼ detector training

- Bag-of Freebies

- **S**: Eliminate grid sensitivity
- **M**: Mosaic data augmentation
- **IT**: IoU threshold
- **GA**: Genetic algorithms
- **LS**: Class label smoothing
- **CBN**: CmBN
- **CA**: Cosine annealing scheduler
- **DM**: Dynamic mini-batch size
- **OA**: Optimized Anchors

Table 4: Ablation Studies of Bag-of-Freebies. (CSPResNeXt50-PANet-SPP, 512x512).

S	M	IT	GA	LS	CBN	CA	DM	OA	loss	AP	AP ₅₀	AP ₇₅
									MSE	38.0%	60.0%	40.8%
✓									MSE	37.7%	59.9%	40.5%
	✓								MSE	39.1%	61.8%	42.0%
		✓							MSE	36.9%	59.7%	39.4%
			✓						MSE	38.9%	61.7%	41.9%
				✓					MSE	33.0%	55.4%	35.4%
					✓				MSE	38.4%	60.7%	41.3%
						✓			MSE	38.7%	60.7%	41.9%
							✓		MSE	35.3%	57.2%	38.0%
✓									GIoU	39.4%	59.4%	42.5%
✓									DIoU	39.1%	58.8%	42.1%
✓									CIoU	39.6%	59.2%	42.6%
✓	✓	✓	✓						CIoU	41.5%	64.0%	44.8%
	✓	✓	✓	✓				✓	CIoU	36.1%	56.5%	38.4%
✓	✓	✓	✓	✓				✓	MSE	40.3%	64.0%	43.1%
✓	✓	✓	✓	✓				✓	GIoU	42.4%	64.4%	45.9%
✓	✓	✓	✓	✓				✓	CIoU	42.4%	64.4%	45.9%

S, M, IT, GA, OA, GIoU/CIoU를 적용했을때 가장 좋은 성능을 보임

- Bag-of Specials

Table 5: Ablation Studies of Bag-of-Specials. (Size 512x512).

Model	AP	AP ₅₀	AP ₇₅
CSPResNeXt50-PANet-SPP	42.4%	64.4%	45.9%
CSPResNeXt50-PANet-SPP-RFB	41.8%	62.7%	45.1%
CSPResNeXt50-PANet-SPP-SAM	42.7%	64.6%	46.3%
CSPResNeXt50-PANet-SPP-SAM-G	41.6%	62.7%	45.0%
CSPResNeXt50-PANet-SPP-ASFF-RFB	41.1%	62.6%	44.4%

- **SPP**: Spatial Pyramid Pooling
- **RFB**: Receptive Field Block
- **SAM**: Spatial Attention Module
- **ASFF**: Adaptively Spatial Feature Fusion
- **G**: Gaussian YOLO

SPP-SAM을 적용했을때 가장 좋은 성능을 보임

- classifier vs detection

Table 6: Using different classifier pre-trained weightings for detector training (all other training parameters are similar in all models) .

Model (with optimal setting)	Size	AP	AP ₅₀	AP ₇₅
CSPResNeXt50-PANet-SPP	512x512	42.4	64.4	45.9
CSPResNeXt50-PANet-SPP (BoF-backbone)	512x512	42.3	64.3	45.7
CSPResNeXt50-PANet-SPP (BoF-backbone + Mish)	512x512	42.3	64.2	45.8
CSPDarknet53-PANet-SPP (BoF-backbone)	512x512	42.4	64.5	46.0
CSPDarknet53-PANet-SPP (BoF-backbone + Mish)	512x512	43.0	64.9	46.5

꼭 classification에서 성능이 좋다고해서 detection에서도 좋은 것은 아님

- mini-batch

Table 7: Using different mini-batch size for detector training.

Model (without OA)	Size	AP	AP ₅₀	AP ₇₅
CSPResNeXt50-PANet-SPP (without BoF/BoS, mini-batch 4)	608	37.1	59.2	39.9
CSPResNeXt50-PANet-SPP (without BoF/BoS, mini-batch 8)	608	38.4	60.6	41.6
CSPDarknet53-PANet-SPP (with BoF/BoS, mini-batch 4)	512	41.6	64.1	45.0
CSPDarknet53-PANet-SPP (with BoF/BoS, mini-batch 8)	512	41.7	64.2	45.2

ResNeX와 다르게 Darknet53에서는 mini-batch를 작게해도 성능은 거의 변동없이 보장됨

▼ Conclusion

YOLOv3을 개선한 더 빠르고 정확한 모델을 제안한 것

Single GPU환경에서 사용가능한 모델